

Fundamentals of Neural Networks: Gradient Descent for Linear Regression

Didi Adawi 

dsadaw14@gmail.com

April 12, 2026

1 Introduction

1.1 Aim

The aim of this exploration is to use mathematical concepts to explain the inner workings of how neural networks learn/are trained, with a numerical method on a simple example of learning a curve, or more commonly known as a linear regression. Learning a curve means finding the line that best follows the trend of the data by gradually adjusting its position and slope. Training will be performed on a known first-degree polynomial with 11 points chosen. The first-degree polynomial is first regressed to allow for a visual representation, easier explanation, and verification of results. A more complex polynomial will then be regressed to demonstrate the versatility of gradient descent.

1.2 Foundations

This exploration will assume an understanding of basic differential calculus and basic linear algebra. Topics such as optimization and derivatives, to include derivatives of multivariate¹ functions will be expanded upon. This exploration will build on existing knowledge and introduce prerequisite knowledge of neural networks and multivariable calculus before applying it in practice to perform linear regressions.

¹A function having more than 1 variable as an input.

2 Prerequisites

2.1 Vectors and Vectorspaces

A real vector $\vec{v}$ with n components can be written as,

$$\vec{v} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix}.$$

A vector is defined as an n -tuple² of real and is said to exist in the vector-space $\mathbb{R}^n$. n can also be referred to as the dimension of the vector.

2.2 Neural Networks

A neural network is a mathematical model that takes input values, processes them, and produces an output. During training, it improves its output by adjusting internal values called parameters so that its predictions better match the data. In this exploration, the network is used in a very simple way: it learns the best-fitting straight line for a set of points.

Parameters play a big role in the outputs a neural network produces. Parameters are the weights and biases, similar to coefficients and constant terms in a polynomial, respectively, which determine how much an input contributes to the output.

A dataset is a set of inputs and respective outputs. When a neural network is trained on a dataset, its parameters are optimized to achieve the desired outputs for a given set of inputs, resulting in a trained model. These trained models are dataset-specific, so the same network trained on different datasets will yield different outputs.

Mathematically, neural networks can be written as a family of functions parameterized by a parameter vector $\vec{\theta}$ [1]:

$$f_{\theta} : \mathbb{R}^m \rightarrow \mathbb{R}^n. \quad 3$$

A neural network is written as f_{θ} because it is not just one fixed function. It is a function whose behavior depends on a set of adjustable values, called parameters, collected in a vector $\vec{\theta}$. Changing $\vec{\theta}$ changes the output of the function. $\vec{\theta}$ is a vector of parameters and θ is a specific choice of m parameters(weights/coefficients), which is also the number of inputs, and n is the number of outputs, where $m, n \in \mathbb{Z}^+$.

For the specific case of using a very simplified network to learn a specific curve of degree 1 polynomial, the parameters are the coefficients of each term in the polynomial. The family

²An ordered list of elements, an example of which is $(a_1, a_2) \neq (a_2, a_1)$.

³The notation of: $\mathbb{R}^m \rightarrow \mathbb{R}^n$ is used to define how the function maps inputs to outputs. An $\mathbb{R}^2 \rightarrow \mathbb{R}^1$ function maps 2 real inputs to 1 real output, an example of which could be $f(x, y) = x + y$.

of functions of such a neural network looks like

$$f_{\theta}(x) = \theta_1 x + \theta_0, \text{ with } f_{\theta} : \mathbb{R}^1 \rightarrow \mathbb{R}^1 \text{ and } \vec{\theta} = \begin{pmatrix} \theta_1 \\ \theta_0 \end{pmatrix}, \quad (1)$$

where f_{θ} is the estimate of y for a given x , $\vec{\theta}$ being the weights and biases in the parameter vector.

2.3 Partial Derivatives

A univariate⁴ function such as

$$y = f(x) = x^2,$$

produces a 2D curve, the of which only differs with respect x ; therefore, the only derivative with respect to an input variable is $\frac{df}{dx}$. However, a multivariate function such as

$$z = f(x, y) = -x^2 - y^2, \text{ with } f : \mathbb{R}^2 \rightarrow \mathbb{R}, \quad (2)$$

describes all points whose height z is determined by the values of x and y and defines a surface in 3D, see Figure 1, the change of which differs with respect to both x and y . The two derivatives would then be $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$. ∂ is used for partial derivatives to clarify that all other variables are held constant. The partial derivatives of Equation 2 are derived by:

$$\begin{aligned} \frac{\partial f}{\partial x} &= \frac{\partial}{\partial x} (-x^2 - y^2) \\ &= \frac{\partial}{\partial x} (-x^2) + \frac{\partial}{\partial x} (-y^2) \\ &= -2x + 0 \\ &= -2x \end{aligned}$$

The derivation of $\frac{\partial f}{\partial y}$ proceeds similarly, so the intermediate steps are omitted, and hence, the partial derivatives are $\frac{\partial f}{\partial x} = -2x$ and $\frac{\partial f}{\partial y} = -2y$.

2.4 Gradients

Consider a function of the form $f : \mathbb{R}^n \rightarrow \mathbb{R}$. The gradient ∇f is defined as the vector at point $(x_1, x_2, \dots, x_n)$, which points in the direction of greatest increase on the surface of f while $\nabla f \neq \vec{0}$ [2] and can be written as:

$$\nabla f(x_1, x_2, \dots, x_n) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(x_1, x_2, \dots, x_n) \\ \frac{\partial f}{\partial x_2}(x_1, x_2, \dots, x_n) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x_1, x_2, \dots, x_n) \end{pmatrix}. \quad (3)$$

⁴A function having only 1 variable.

For a function of two real input variables, it is the steepest ascent on the surface $z = f(x, y)$, and in the form

$$\nabla f(x, y) = \begin{pmatrix} \frac{\partial f}{\partial x}(x, y) \\ \frac{\partial f}{\partial y}(x, y) \end{pmatrix}$$

In Figure 1, $\nabla f(1, 0.5)$ is drawn in blue. Since the function is concave⁵, commonly referred to as "concave down" in most Calculus I courses, the gradient vector points in the direction of steepest ascent; therefore, pointing to the nearest maximum of the function.

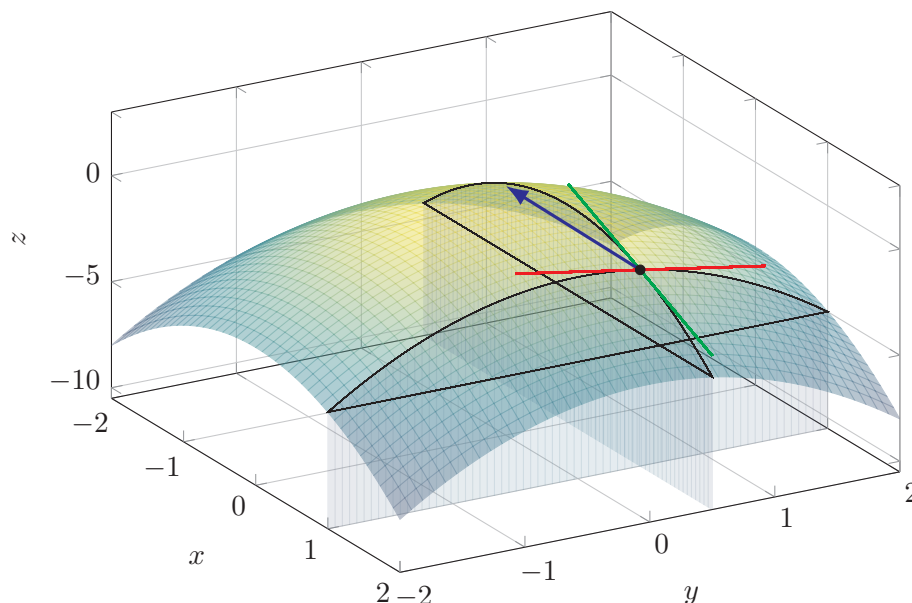


Figure 1: Surface $z = f(x, y) = -(x^2 + y^2)$. Tangent lines to the coordinate traces at $(1, 0.5)$ with slopes $\frac{\partial f}{\partial x}(1, 0.5)$ (green) and $\frac{\partial f}{\partial y}(1, 0.5)$ (red). The gradient vector $\nabla f(1, 0.5)$, in blue, is displayed on the tangent plane to aid visualization. (Self-made with Pgfplots.)

2.5 The Loss Function

For a neural network, a loss function quantifies the error from the expected value for a given input on a dataset. The loss function utilized for this exploration will be the sum of squared residuals,⁶ henceforth SSR, because squaring the error, which makes all losses non-negative, higher errors are penalized more, and most importantly, it is easily differentiable, which allows for minimization with derivatives. For any linear regression, the loss function is convex⁷, commonly referred to as "concave up" in most Calculus I courses, because error

⁵A function with many input variables for which the line segment joining any two points on the graph is never above the graph [3].

⁶Vertical distance between estimate and expected value, in the form $y - \hat{y}$, where for a given input x , y is the expected value and $\hat{y}$ is the estimate.

⁷A function with many input variables for which the line segment joining any two points on the graph is never below the graph [3]

is linear and is squared, which produces a parabolic shape, and loss is non-negative. Due to the convexity, any gradient vector will point in the direction of greatest positive change in f ; however going in the direction of the gradient vector will lead to divergence, because convex functions do not have maxima. To achieve minimization, $-\nabla f$ is taken to find the direction of greatest negative change in f , which can lead to convergence.

The input variables of the loss function $\mathcal{L}$ are the parameters in the parameter vector $\vec{\theta}$, so the general loss function for a neural network trained on the dataset D is defined as

$$\mathcal{L}(\theta_n, \theta_{n-1}, \dots, \theta_0) = \sum_{i \in D} (y_i - (\theta_0 + \theta_1 x_i + \dots + \theta_n x_i^n))^2,$$

simplifying,

$$\mathcal{L}(\theta_n, \theta_{n-1}, \dots, \theta_0) = \sum_{i \in D} (y_i - f_{\theta}(x_i))^2, \quad (4)$$

where i is the index of a length s dataset, $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_s, y_s)\}$ and f_{θ} is the family of functions, which represents the neural network parameterized by $\vec{\theta}$.

For this specific exploration, the loss function will be:

$$\mathcal{L}(\theta_1, \theta_0) = \sum_{i \in D} (y_i - (\theta_1 x_i + \theta_0))^2. \quad (5)$$

Since the loss function measures error, for the loss function in this exploration, minimizing it yields the θ_1 and θ_0 , for which loss is minimized and where the curve most resembles the relationship of a dataset. Also, by the definition of convex, all minima are global minima, so every minimum results in a curve that most closely follows the relationship of the dataset.

2.6 Backpropagation & Gradient Descent

Gradient descent is the numerical process of minimizing $\mathcal{L}$ with 3 steps: the forward pass, backward pass, and updating the parameter vector $\vec{\theta}$. The forward pass computes the total loss of f_{θ} on a dataset. The backward pass calculates how and by how much to change the parameters. The proportion by which each parameter should be adjusted is found by computing $-\nabla \mathcal{L}(\theta_1, \theta_0)$. The last step is updating the parameters in the following form:

$$\vec{\theta}_{t+1} = \vec{\theta}_t - \alpha \nabla_{\vec{\theta}} \mathcal{L}(\theta_t), \quad (6)$$

where $\vec{\theta}$ is the current parameter vector, $\vec{\theta}_{t+1}$ is the new parameter vector, α is the learning rate, and $\nabla_{\vec{\theta}} \mathcal{L}(\theta_t)$, equivalent to $\nabla_{\vec{\theta}} \mathcal{L}(\theta_1, \theta_0)$, the loss with current parameters. The reason the scaled gradient vectors are subtracted from the current parameter vector is that $-\nabla_{\vec{\theta}} \mathcal{L}(\theta_t)$ is taken due to the loss function being convex.

Learning rate is a scalar multiple of $\nabla_{\vec{\theta}} \mathcal{L}(\theta_t)$ typically with $\alpha \in (0, 1)$. It is also one of the hyperparameters of the algorithm; settings which are chosen prior to or during training, must be determined outside of the learning algorithm[1]. Learning rate is used because for

an $\alpha \geq 1$, gradient descent could be unstable and may cause a failure to converge over many iterations. The process outlined in Equation 6 is theoretically repeated until $\nabla_{\vec{\theta}} \mathcal{L}(\theta_t) = 0$, but this case rarely exactly happens because this is a numerical method, meaning $\nabla_{\vec{\theta}} \mathcal{L}(\theta_t)$ only converges to 0 with infinite iterations. In most cases, $\nabla_{\vec{\theta}} \mathcal{L} \rightarrow \vec{0}$ with many iterations.

A more intuitive explanation for newcomers can be illustrated with a neural network of only 2 parameters. The graph of a loss function for a neural network with 2 parameters can be visualized as a 3-dimensional surface. For neural networks with more than 2 parameters, the resultant loss surface is a hypersurface, which exists in $\mathbb{R}^{n+1}$, where n is the number of parameters, and cannot be easily visualized[4]. On a graph, gradient descent can be thought of with the analogy of a rolling a ball down from the top of a hill in a hillscape and the place where it is at rest, is most often⁸ the lowest point in the valley, see Figure 2 and Figure 5.

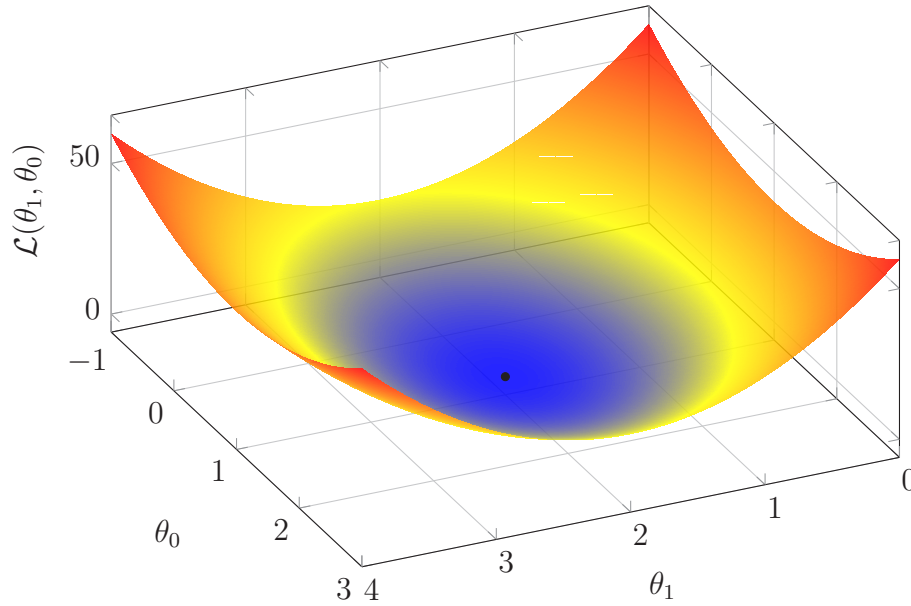


Figure 2: Surface $z = \mathcal{L}(\theta_t) = \sum_i (y_i - f_{\theta}(x_i))^2$. Visualization of Equation 5. The black dot at the minimum represents the ball at rest and the bottom of hillscape. (Self-made with Pgfplots.)

To find $\nabla_{\vec{\theta}} \mathcal{L}(\theta_t)$, the sum of all gradient vectors, the partial derivatives must be calculated. The general partial derivative of loss with respect to θ_n can be calculated by using the chain

⁸Non-convex and complex loss functions can have multiple minima, with not all being global minima.

rule, in this case also called backpropagation:

$$\begin{aligned}
\frac{\partial}{\partial \theta_n} (\mathcal{L}(\theta_t)) &= \frac{\partial}{\partial \theta_n} \left(\sum_i (y_i - f_{\theta_t}(x_i))^2 \right) \\
&= \frac{\partial}{\partial \theta_n} \left(\sum_i (y_i - (\theta_n x_i^n + \dots + \theta_0))^2 \right) \\
&= \sum_{i \in D} \frac{\partial}{\partial \theta_n} ((y_i - \theta_n x_i^n - \dots - \theta_0)^2) \\
&= \sum_{i \in D} 2 \cdot (y_i - \theta_n x_i^n - \dots - \theta_0) \cdot (-x_i^n)
\end{aligned}$$

Let $\Delta y_i = y_i - \theta_n x_i^n - \dots - \theta_0$,

$$= - \sum_{i \in D} 2 \Delta y_i x_i^n$$

With the general definition of the partial derivative of loss with respect to any parameter θ_n , $\nabla \mathcal{L}(\theta_t)$ can be defined as:

$$\nabla_{\vec{\theta}} \mathcal{L}(\theta_t) = \begin{pmatrix} - \sum_{i \in D} 2 \Delta y_i \cdot x_i^n \\ - \sum_{i \in D} 2 \Delta y_i \cdot x_i^{n-1} \\ \vdots \\ - \sum_{i \in D} 2 \Delta y_i \end{pmatrix},$$

and for the case of a first-degree polynomial:

$$\nabla_{\vec{\theta}} \mathcal{L}(\theta_t) = \begin{pmatrix} - \sum_{i \in D} 2 \Delta y_i \cdot x_i \\ - \sum_{i \in D} 2 \Delta y_i \end{pmatrix}. \tag{7}$$

3 Known First-Order Polynomial

3.1 Target Polynomial & Dataset

The selected curve to which a neural network should arrive to is

$$f(x) = 2x + 1 \tag{8}$$

In most cases, the target will not be known prior to solving; however to measure error from the expected value, points will be drawn from Equation 8 and used as the training dataset.

The following 11 points, which satisfy Equation 8, are used for the dataset

$$D = \{(0, 1), (1, 3), (2, 5), (3, 7), (4, 9), (5, 11), (6, 13), (7, 15), (8, 17), (9, 19), (10, 21)\} \quad (9)$$

3.2 Neural Network & Loss Functions

Recall that the network function from Equation 1 for a given set of parameters $\vec{\theta}$ is

$$f_{\theta}(x) = \theta_1 x + \theta_0$$

with a corresponding loss function from Equation 5 of

$$\mathcal{L}(\theta_t) = \sum_{i=0}^{10} (y_i - (\theta_1 x_i + \theta_0))^2,$$

due to the 11 points used, which can be simplified to and visualized as Figure 2

$$\mathcal{L}(\theta_t) = \sum_{i=0}^{10} (y_i - f_{\theta}(x_i))^2. \quad (10)$$

3.3 Gradient Descent

Recall the equation Equation 6 for updating the parameter vector $\vec{\theta}_t$ is:

$$\vec{\theta}_{t+1} = \vec{\theta}_t - \alpha \nabla_{\vec{\theta}} \mathcal{L}(\theta_t).$$

The initial $\vec{\theta}_t$ is arbitrarily chosen, as it's choice only affects the number of iterations of gradient descent and minimization required, and will be $\begin{pmatrix} 0 \\ 0 \end{pmatrix}$, with $\alpha = 10^{-3}$. This α was obtained experimentally by using different powers of 10 in Appendix A; higher powers of 10 cause divergence to ∞ .

Substituting for chosen values, the first iteration will be

$$\vec{\theta}_{t+1} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \begin{pmatrix} 0.001 \cdot \sum_{i \in D} 2\Delta y_i \cdot x_i \\ 0.001 \cdot \sum_{i \in D} 2\Delta y_i \end{pmatrix},$$

with all further iterations being in the form

$$\vec{\theta}_{t+1} = \vec{\theta}_t + \begin{pmatrix} 0.001 \cdot \sum_{i \in D} 2\Delta y_i \cdot x_i \\ 0.001 \cdot \sum_{i \in D} 2\Delta y_i \end{pmatrix}. \quad (11)$$

One iteration of gradient descent will be performed manually to show proof of concept, with the remaining iterations being performed by a Python script Appendix A

Initial loss is:

$$\mathcal{L}(\theta_t) = \sum_{i=0}^{10} (y_i - f_{\theta}(x_i))^2,$$

substituting and expanding for dataset D Equation 9,

$$\begin{aligned}\mathcal{L}(0,0) &= 1^2 + 3^2 + 5^2 + 7^2 + 9^2 + 11^2 + 13^2 + 15^2 + 17^2 + 19^2 + 21^2 \\ &= 1 + 9 + 25 + 49 + 81 + 121 + 169 + 225 + 289 + 361 + 441 \\ &= 1771.\end{aligned}$$

Calculating the sum of gradient vectors $\nabla_{\theta}\mathcal{L}(\theta_t)$ and $\vec{\theta}_t$, the partial derivative $\frac{\partial \mathcal{L}}{\partial \theta_1}$ is calculated:

$$\sum_{i=0}^{10} 2\Delta y_i x_i = 2(1 \cdot 0 + 3 \cdot 1 + 5 \cdot 2 + 7 \cdot 3 + 9 \cdot 4 + 11 \cdot 5 + 13 \cdot 6 + 15 \cdot 7 + 17 \cdot 8 + 19 \cdot 9 + 21 \cdot 10),$$

and the same is done to calculate $\frac{\partial \mathcal{L}}{\partial \theta_0}$:

$$\sum_{i=0}^{10} 2y_i = 2(1 + 3 + 5 + 7 + 9 + 11 + 13 + 15 + 17 + 19 + 21).$$

Therefore,

$$\begin{aligned}\vec{\theta}_{t+1} &= \begin{pmatrix} 0.001 \cdot 2(0 + 3 + 10 + 21 + 36 + 55 + 78 + 105 + 136 + 171 + 210) \\ 0.001 \cdot 2(1 + 3 + 5 + 7 + 9 + 11 + 13 + 15 + 17 + 19 + 21) \end{pmatrix} \\ &= \begin{pmatrix} 1.65 \\ 0.242 \end{pmatrix},\end{aligned}$$

meaning that after the first iteration, the estimated slope is $\theta_1 = 1.65$, with a percentage difference of 17.5% from the true slope, and the estimated y-intercept is $\theta_0 = 0.242$, with a percentage difference of 75.8% from the true y-intercept. However, as previously mentioned, these initial differences are only the result of choosing an initial $\vec{\theta} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$, which is very close to the target parameters.

Now, using the Python script in Appendix A, the process was repeated for 4000 iterations to obtain the target parameter vector $\vec{\theta}$. This value was obtained experimentally by using different multiples of 1000; the difference between the target and calculated values became negligible between 3000-4000 iterations, to the extent that the script was returning integer parameters (2, 1). The change between iterations was small enough and parameters were close to the true values, due to limitations in computer precision for representable rational numbers.

$$\vec{\theta} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}.$$

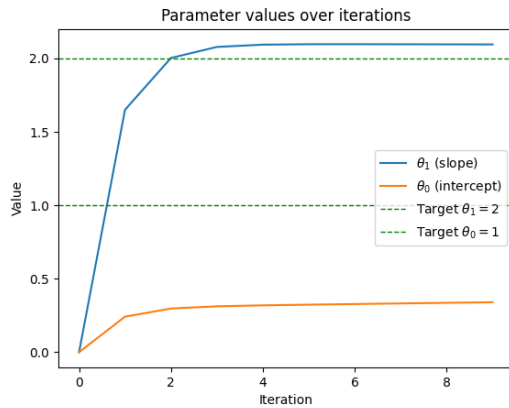


Figure 3: θ_1 graphed in blue and θ_0 graphed in orange, against 10 iterations. (Self-made with Matplotlib Appendix A.)

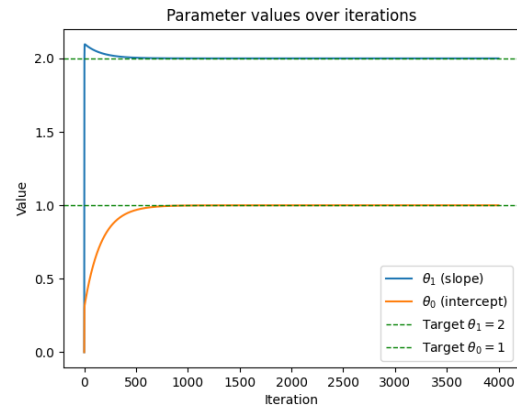


Figure 4: θ_1 graphed in blue and θ_0 graphed in orange, against 4000 iterations. (Self-made with Matplotlib Appendix A.)

About every 50-*th* point throughout the 4000 iterations were taken and graphed alongside the loss function graph, and the visual of the ball rolling down a hill explained in Subsection 2.6 becomes more apparent as the points start to approach the minima of the loss function.

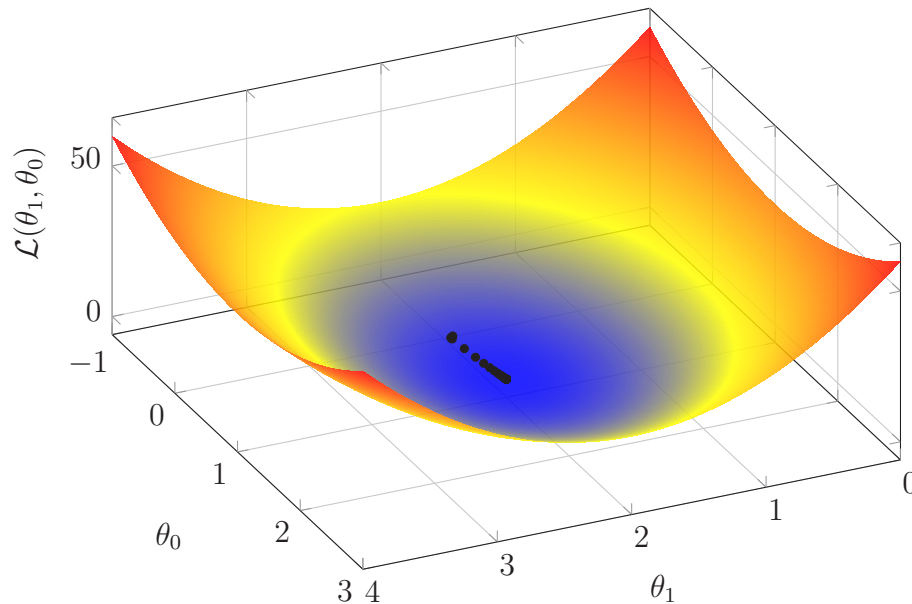


Figure 5: Surface $z = \mathcal{L}(\theta_1, \theta_0) = \sum_{i=0}^{10} (y_i - f_{\theta}(x_i))^2$. Visualization of Equation 10. (Self-made with Pgfplots.)

This result means that the estimate by repeating 4000 iterations of gradient descent for the first-degree polynomial is:

$$f_{\theta} = 2x + 1, \vec{\theta} = \begin{pmatrix} 2 \\ 1 \end{pmatrix},$$

which is exactly what the target curve is. This shows that neural networks trained with

gradient descent and backpropagation can be used to learn first-degree polynomials. With this foundation, neural networks can be used to learn n -degree polynomials or more complex relationships.

4 Fifth Order Polynomial

4.1 Polynomial Generation & Dataset

To demonstrate the versatility of this method, a third-degree polynomial was fit to the points that satisfy the equation of a fifth-degree polynomial, the coefficients of which were arbitrarily chosen. The result will then be confirmed using other methods to validate the outcome. The chosen fifth-degree polynomial is:

$$f(x) = 4x^5 - 4x^4 + 7x^3 + 5x^2 - 3x - 2. \quad (12)$$

Due to the more pronounced fifth-order behavior exhibited in the range $y \in [-1, 1]$, 21 evenly-spaced points, which satisfy Equation 12 in the range $[-1, 1]$ will be drawn:

$$\begin{aligned} D = \{ & (-1.000, -9.000), (-0.900, -5.339), (-0.800, -2.933), (-0.700, -1.484), \\ & (-0.600, -0.741), (-0.500, -0.500), (-0.400, -0.591), (-0.300, -0.881), \\ & (-0.200, -1.264), (-0.100, -1.657), (0.000, -2.000), (0.100, -2.243), \\ & (0.200, -2.349), (0.300, -2.284), (0.400, -2.013), (0.500, -1.500), \\ & (0.600, -0.695), (0.700, 0.463), (0.800, 2.056), (0.900, 4.191), (1.000, 7.000) \}. \end{aligned} \quad (13)$$

4.2 Neural Network & Loss Function

Similarly to Subsection 3.2, the family of functions that describe the neural network in this scenario is:

$$f_{\theta}(x) = \theta_3 x^3 + \theta_2 x^2 + \theta_1 x + \theta_0, \quad (14)$$

and the corresponding SSR loss function of:

$$\mathcal{L}(\theta_t) = \sum_{i=0}^{20} (y_i - f_{\theta}(x_i))^2 \quad (15)$$

4.3 Gradient Descent

For this example, there was not a fixed number of iterations of gradient descent run. Instead, iterations were run until all components of the $\nabla_{\vec{\theta}} \mathcal{L}(\theta_t)$ vector were less than the threshold of 10^{-9} . All the individual steps will be omitted as they are trivial and unnecessary. The

result from the numerical regression is the parameter vector of:

$$f_{\theta}(x) = 11.8x^3 + 1.26x^2 - 4.12x - 1.59, \text{ clearly with } \vec{\theta} = \begin{pmatrix} 11.8 \\ 1.26 \\ -4.12 \\ -1.59 \end{pmatrix} \quad (16)$$

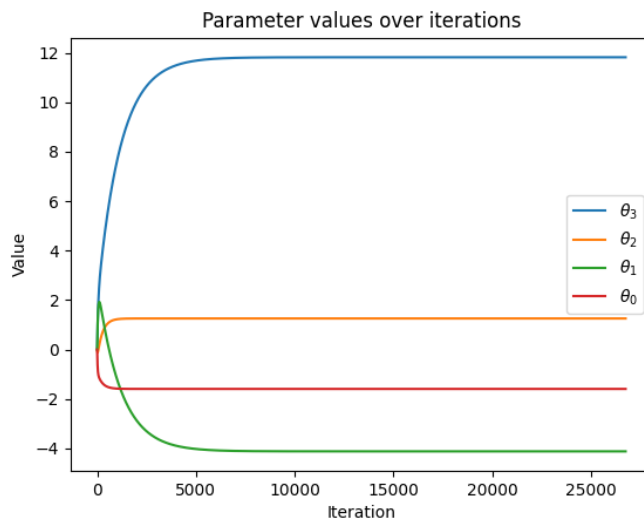


Figure 6: All parameters, θ_n , graphed against iteration count (Self-made with Matplotlib Appendix B)

4.4 Coefficient of Determination

R^2 is called the coefficient of determination and can be interpreted as the proportion of the total variability in the response variable the output that can be accounted for by the set of inputs[5]. According to Chatterjee and Hadi, it is defined as:

$$R^2 = 1 - \frac{SSR}{SST} = 1 - \frac{\sum_{i \in D} (y_i - \hat{y}_i)^2}{\sum_{i \in D} (y_i - \bar{y})^2}.$$

However, it does have limitations, as no single measure will be adequate for describing the usefulness of a regression model for different applications. Some limitations and considerations are that a high coefficient of determination indicates that useful predictions can be made using the model and that a high coefficient of determination implies that the estimated regression is a good fit[6].

The coefficient of determination will be used to determine the difference between the predicted line using "Scipy," a Python library used for data analysis, which uses an analytical method and yields a very similar result, and the numerical gradient descent method. Com-

paring the two values:

$$R_{SP}^2 = 0.98278\dots$$

$$R_{GD}^2 = 0.98278\dots,$$

it can be observed that they are identical⁹, again suggesting that with an increasing number of iterations, the numerical solution converges to the analytical solution.

5 Hyperparameter Tuning

Hyperparameters are values, which mostly affect time to convergence and quality of minima, need to me determined experimentally through trials, and are usually dataset-specific. A very common theme in hyperparameters is compromise, usually between time and quality.

5.1 Gradient Threshold

A gradient threshold means that the magnitude of all gradients $\frac{\partial \mathcal{L}}{\partial(\theta_1)}, \frac{\partial \mathcal{L}}{\partial(\theta_2)}, \dots, \frac{\partial \mathcal{L}}{\partial(\theta_n)}$, must be less than the threshold value, as that would imply that any additional iteration would have a "negligible" effect on minimizing loss. Here the notion of significance is determined by the threshold as some may value minimizing loss more than others.

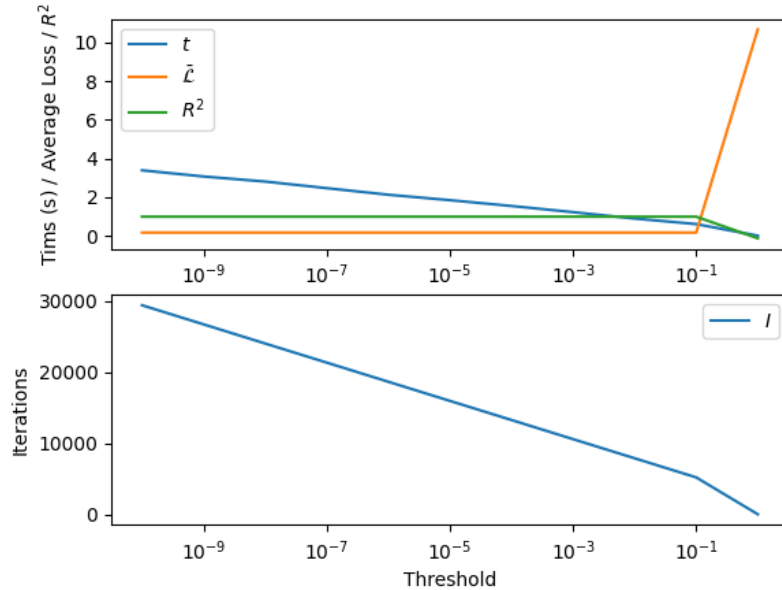


Figure 7: Time, t , in seconds, average loss, $\bar{\mathcal{L}}$, coefficient of determination, R^2 , and number of iterations, I , graphed against threshold value. Any appearance that R^2 decreases is purely a visual anomaly¹⁰.

⁹Both values were 0.9827820827316149; however more than 5 digits were not reported for brevity.

As with most things, there is a trade-off. There is a balance between the threshold value and the number of iterations to train; consequently, a balance in the time taken. An exploration into how the threshold affects the number of iterations taken for the model in Section 4 yielded Figure 7. As seen in the top part of the graph, after a certain point, around 10^{-1} , the average loss, $\bar{\mathcal{L}}$, and coefficient of determination, R^2 , are mostly stagnant. Both time and the number of iterations exhibit a negative logarithmic relationship against the threshold value, with both increasing roughly linearly with the number of decimal orders of magnitude by which the threshold is reduced. Therefore, discretion must be taken to consider the utility of marginally better training results compared to the time taken to train the model.

5.2 Gradient Descent Variations

The optimization algorithm is also considered a hyperparameter. The foundation for many optimization algorithms is gradient descent with additional features to improve time to convergence, quality of minima, etc.

5.2.1 Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent, henceforth SGD, is a variant of regular gradient descent, wherein a small batch of the training dataset, usually randomly selected for each batch, is used to estimate the true sum of gradient vectors. Using a smaller subset of the full dataset to estimate the gradient vector results in less efficient¹¹ gradient vectors; however, the time for each iteration is much less, depending on the size of each batch. SGD is increasingly useful with larger datasets, where each iteration takes a significant amount of time due to the sheer volume of training data, and the time saved by completing the computation on the full dataset outweighs the additional time required to run more iterations.

5.2.2 Momentum

This section will be relatively short as this is an abstract concept, which would require a more lengthy explanation, outside of what is reasonable for an introductory explanation of the training of machine learning models. Momentum is implemented in many optimization algorithms and is meant to mimic the notion of momentum in physics. Momentum adds an additional property of "velocity" besides the position of the current parameter vector, which allows the algorithm to "escape" relatively shallow/false minima on the loss surface, by using the velocity accumulated from the descent to determine if minima are sufficiently deep.

¹⁰An analysis was done to confirm that with a decreasing threshold value, the change in R^2 is non-negative.

¹¹Since the estimated gradient vector does not use all the data, it will not result in the most optimal path to convergence, instead, possibly causing some oscillations.

5.3 Learning Rate

Learning rate, α , can affect the training in two primary ways: time and outcome. A smaller learning rate causes time to increase as each "step length" at each iteration is smaller, requiring more steps, and in turn, more computations. However, as suggested in previous sections, too high of a learning rate can cause oscillation, which can result in divergence. The balance of time and the risk of divergence often requires the perfect learning rate to be experimentally.

5.4 Adam

Adam is an optimization algorithm that incorporates all the aforementioned ideas and more, like an adaptive learning rate, into one algorithm. It, or its variant with weight decay, AdamW, is one of the most widely used optimization algorithms today. It is straightforward to implement, computationally efficient, has low memory requirements, and is well-suited for large-scale problems in terms of data and/or parameters [7].

6 Conclusion

To reiterate, the aim of this exploration was to apply mathematical concepts, such as calculus and vectors, to obtain numerical solutions for linear regression. The use of gradient descent is explained to provide a basic foundation for implementing the algorithm in more complex neural networks with non-linear behavior, which typically precludes the use of an analytical method.

Some of the limitations of this method are that, for the specific case of linear regressions, gradient descent is overly complex, as a closed-form analytical solution exists, which is much more efficient to calculate than the numerical solution from gradient descent. However, this method shines for datasets that exhibit more complex and non-linear relationships. The method is very versatile and scalable, but is relatively slow for simpler relationships and scales better with complex relationships.

The choosing of the learning rate was also another limitation, because as mentioned, it was only determined experimentally. This could lead to longer iteration counts as the learning rate could be further optimized to increase computational efficiency. One approach is an adaptive learning rate, similar to the method used in Adam, in which the learning rate changes over time to result in faster convergence by initially having a higher learning rate and then shrinking it down to ensure convergence.

Currently, the loss function sums the loss of every point in the dataset, which works for this implementation, because the loss is relatively simple to compute and there aren't many points. However, with larger datasets, calculating the loss at each step becomes very time-consuming. The fix to this would be the aforementioned SGD [8], where only a sample of some of the points in the dataset is taken to calculate how to update the parameters. Although not perfect, it is much more efficient computationally and can be a better algorithm, at least

for the initial iterations, where the goal is to find a minimum.

Lastly, some loss functions can have minima, not all of which are global minima, leading to false or non-optimal parameters. This is why more complex algorithms incorporate momentum alongside gradient descent, which continues to follow the metaphor of a ball rolling down a hill. Non-global minima will be shallower, and the momentum of the proverbial ball will allow the ball to exit the non-global minimum; however, for global minima, the ball will not have enough energy to escape, and will become stationary.

Gradient descent is the method on which most advanced algorithms are built on, and the knowledge gained throughout the course of this exploration is the foundation of neural networks and should allow the reader to explore creating their own models.

References

- [1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [2] Stephen P. Boyd and Lieven Vandenberghe. *Convex optimization* Stephen Boyd (*Department of Electrical Engineering, Stanford University*), Lieven Vandenberghe (*Electrical Engineering Department, University of California, Los Angeles*). Cambridge University Press, 2023. URL https://web.stanford.edu/~boyd/cvxbook/bv_cvxbook.pdf.
- [3] Martin J. Osborne. Mathematical methods for economic theory: 3.3 Concave and convex functions of many variables, September 2024. URL <https://mjo.osborne.economics.utoronto.ca/index.php/tutorial/index/1/cvn/t>.
- [4] Wilfred Kaplan. Advanced calculus 5th edition, Jul 2002.
- [5] Samprit Chatterjee and Ali S. Hadi. *Regression Analysis by Example: Chatterjee/Regression*. Wiley Series in Probability and Statistics. John Wiley & Sons, Inc., Hoboken, NJ, USA, September 2006. ISBN 978-0-470-05546-5 978-0-471-74696-6. doi: 10.1002/0470055464. URL <http://doi.wiley.com/10.1002/0470055464>.
- [6] M.H. Kutner. *Applied Linear Statistical Models*. McGraw-Hill international edition. McGraw-Hill Irwin, 2005. ISBN 978-0-07-112221-4. URL <https://books.google.com/books?id=0xqCAAAACAAJ>.
- [7] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017. URL <https://arxiv.org/abs/1412.6980>.
- [8] ML - Stochastic Gradient Descent (SGD), February 2019. URL <https://www.geeksforgeeks.org/machine-learning/ml-stochastic-gradient-descent-sgd/>.

A First Order Polynomial Source Code

This code is part of a bigger repository seen [here](#).

Listing 1: Statistics Calculations for First Order Polynomial

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 points = [
5     (0, 1),
6     (1, 3),
7     (2, 5),
8     (3, 7),
9     (4, 9),
10    (5, 11),
11    (6, 13),
12    (7, 15),
13    (8, 17),
14    (9, 19),
15    (10, 21)
16 ]
17 points = np.array(points)
18
19 coefficients = [
20     0, # theta_1
21     0 # theta_0
22 ]
23
24 coefficients = np.array(coefficients)
25
26 alpha = 0.001
27
28 def coef_poly(coefficients, x):
29     return coefficients[0]*x + coefficients[1]
30
31 def forward_prop():
32     tss = 0
33     for point in points:
34         f_theta_x = coef_poly(coefficients, point[0])
35         square_error = (point[1]-f_theta_x)**2
36         tss += square_error
37     return tss
38
39 def back_prop_sum_all():
40     grad = np.zeros(2)
41     for x, y in points:
42         err = (y - coef_poly(coefficients, x))
```

```

43     grad += -2 * err * np.array([x, 1.0])
44     return grad
45
46 def minimize():
47     global coefficients
48     coefficients = coefficients - alpha*back_prop_sum_all()
49
50 n_iters = 4000
51 theta1_hist = []
52 theta0_hist = []
53 loss_hist = []
54
55 for i in range(n_iters):
56     loss_hist.append(forward_prop())
57     theta1_hist.append(coefficients[0])
58     theta0_hist.append(coefficients[1])
59     minimize()
60
61 iters = np.arange(n_iters)
62
63 plt.plot(iters, theta1_hist, label=r'$\theta_1$ (slope)')
64 plt.plot(iters, theta0_hist, label=r'$\theta_0$ (intercept)')
65
66 plt.axhline(2, linestyle='--', linewidth=1, color="green",
67             label=r'Target $\theta_1 = 2$')
68 plt.axhline(1, linestyle='--', linewidth=1, color="green",
69             label=r'Target $\theta_0 = 1$')
70 plt.xlabel("Iteration")
71 plt.ylabel("Value")
72 plt.title("Parameter values over iterations")
73 plt.legend()
74 plt.savefig("fop")
75 plt.show()

```

B Fifth Order Polynomial Source Code

Listing 2: Statistics Calculations for Fifth Order Polynomial

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 import numpy as np
5 from scipy.optimize import curve_fit
6
7 def cubic(x, a, b, c, d):
8     return a*x**3 + b*x**2 + c*x + d

```

```
9
10 def coef_poly(coefficients, x):
11     ttl = 0
12     for i, coeff in enumerate(coefficients):
13         ttl += coeff*x**(len(coefficients)-(i+1))
14     return ttl
15
16 def r2_score_np(y: np.ndarray, y_pred: np.ndarray) -> float:
17     ss_res = np.sum((y - y_pred) ** 2)
18     ss_tot = np.sum((y - np.mean(y)) ** 2)
19     return 1 - ss_res / ss_tot
20
21 _true_coefficients = [
22     4,
23     -4,
24     7,
25     5,
26     -3,
27     -2
28 ]
29
30 points = []
31
32 intervals = 20
33 for i in range(intervals + 1):
34     x = -1+(2/intervals)*i
35     points.append((x, coef_poly(_true_coefficients, x)))
36
37 points = np.array(points)
38
39 x = np.array([point[0] for point in points], dtype=float)
40 y = np.array([point[1] for point in points], dtype=float)
41
42 params, _t = curve_fit(cubic, x, y)
43 print(f"Scipy Coefficient: {params}")
44 print(f"Scipy R^2: {r2_score_np(np.array(y),
45     np.array([coef_poly(params, point[0]) for point in points]))}")
46
47 coefficients = [
48     0,
49     0,
50     0,
51     0
52 ]
53 recent_grad = np.ones(len(coefficients))
54 coefficients = np.array(coefficients)
```

```

55 alpha = 0.001
56
57 def forward_prop():
58     tss = 0
59     for point in points:
60         f_theta_x = coef_poly(coefficients, point[0])
61         square_error = (point[1]-f_theta_x)**2
62         tss += square_error
63     return tss
64
65 def back_prop_sum_all():
66     global recent_grad
67     grad = np.zeros(len(coefficients))
68     for x, y in points:
69         err = (y - coef_poly(coefficients, x))
70         grad += -2 * err * np.array([x**(len(coefficients)-(i+1))
71                                     for i in range(len(coefficients))])
72     recent_grad = grad
73     return grad
74
75 def minimize():
76     global coefficients
77     coefficients = coefficients - alpha*back_prop_sum_all()
78
79 epoch_count = 0
80 theta3_hist = []
81 theta2_hist = []
82 theta1_hist = []
83 theta0_hist = []
84 avg_loss_hist = []
85
86 while (np.any(abs(recent_grad) > 1e-9)):
87     avg_loss_hist.append(forward_prop()/21)
88     theta3_hist.append(coefficients[0])
89     theta2_hist.append(coefficients[1])
90     theta1_hist.append(coefficients[2])
91     theta0_hist.append(coefficients[3])
92     minimize()
93     epoch_count += 1
94
95 iters = np.arange(epoch_count)
96
97 poly_string = ""
98 for i, coeff in enumerate(coefficients):
99     poly_string = poly_string + f"{"+" if i!=0 else
100         "}"({coeff:.2f}x^{len(coefficients)-(i+1)})}"
101
102 print("-----")
103 print(f"GD Poly: {poly_string}")

```

```
100 print(f"GD Coefficients: {coefficients}")
101 print(f"GD R^2: {r2_score_np(np.array([point[1] for point in
    points]), np.array([coef_poly(coefficients, point[0]) for point
    in points]))}")
102 print(f"Epochs: {epoch_count}")
103 print(f"Most recent gradient vector: {recent_grad}")
104
105 plt.plot(iters, theta3_hist, label=r'$\theta_3$')
106 plt.plot(iters, theta2_hist, label=r'$\theta_2$')
107 plt.plot(iters, theta1_hist, label=r'$\theta_1$')
108 plt.plot(iters, theta0_hist, label=r'$\theta_0$')
109
110 plt.xlabel("Iteration")
111 plt.ylabel("Value")
112 plt.title("Parameter values over iterations")
113 plt.legend()
114 plt.savefig("fop")
115 plt.show()
```

C Fifth Order Polynomial Time Comparison Source Code

Listing 3: Statistics Calculations for Fifth Order Polynomial

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 import numpy as np
5 from scipy.optimize import curve_fit
6
7 import time
8
9 def cubic(x, a, b, c, d):
10     return a*x**3 + b*x**2 + c*x + d
11
12 def coef_poly(coefficients, x):
13     ttl = 0
14     for i, coeff in enumerate(coefficients):
15         ttl += coeff*x**(len(coefficients)-(i+1))
16     return ttl
17
18 def r2_score_np(y: np.ndarray, y_pred: np.ndarray) -> float:
19     ss_res = np.sum((y - y_pred) ** 2)
20     ss_tot = np.sum((y - np.mean(y)) ** 2)
21     return 1 - ss_res / ss_tot
```

```

22
23 _true_coefficients = [
24     4,
25     -4,
26     7,
27     5,
28     -3,
29     -2
30 ]
31
32 points = []
33
34 intervals = 20
35 for i in range(intervals + 1):
36     x = -1+(2/intervals)*i
37     points.append((x, coef_poly(_true_coefficients, x)))
38
39 points = np.array(points)
40
41 x = np.array([point[0] for point in points], dtype=float)
42 y = np.array([point[1] for point in points], dtype=float)
43
44 scipy_time_start = time.perf_counter()
45 params, _t = curve_fit(cubic, x, y)
46 scipy_time_end = time.perf_counter()
47 print(f"Scipy Coefficient: {params}")
48 print(f"Scipy R^2: {r2_score_np(np.array(y),
49     np.array([coef_poly(params, point[0]) for point in points]))}")
49 print(f"Time: {scipy_time_end-scipy_time_start}")
50
51 coefficients = [
52     0,
53     0,
54     0,
55     0
56 ]
57 recent_grad = np.ones(len(coefficients))
58 coefficients = np.array(coefficients)
59
60 alpha = 0.001
61
62 def forward_prop():
63     tss = 0
64     for point in points:
65         f_theta_x = coef_poly(coefficients, point[0])
66         square_error = (point[1]-f_theta_x)**2
67         tss += square_error

```

```
68     return tss
69
70 def back_prop_sum_all():
71     global recent_grad
72     grad = np.zeros(len(coefficients))
73     for x, y in points:
74         err = (y - coef_poly(coefficients, x))
75         grad += -2 * err * np.array([x**(len(coefficients)-(i+1))
76                                     for i in range(len(coefficients))])
77     recent_grad = grad
78     return grad
79
80 def minimize():
81     global coefficients
82     coefficients = coefficients - alpha*back_prop_sum_all()
83
84 epoch_count = 0
85 epoch_hist = []
86 time_hist = []
87 loss_hist = []
88 r2_hist = []
89
90
91 powers_of_10 = 10
92 for i in range(powers_of_10+1):
93     iters.append(10**(-i))
94     coefficients=np.zeros(len(coefficients))
95     epoch_count = 0
96     gd_time_start = time.perf_counter()
97     while (np.any(abs(recent_grad) > 10**(-i))):
98         minimize()
99         epoch_count += 1
100     loss_hist.append(forward_prop()/21)
101     gd_time_end = time.perf_counter()
102     epoch_hist.append(epoch_count)
103     time_hist.append(gd_time_end-gd_time_start)
104     r2_hist.append(r2_score_np(np.array([point[1] for point in
105                                     points]), np.array([coef_poly(coefficients, point[0]) for
106                                     point in points])))
107 print("Benchmark done")
108
109
110 plt.subplot(2, 1, 1)
111 plt.plot(iters, time_hist, label=r'$t$')
```



```
112 plt.xscale('log')
113 plt.ylabel(r'Tims (s) / Average Loss /  $R^2$ ')
114 plt.subplot(2, 1, 2)
115 plt.plot(iters, epoch_hist, label=r'$I$')
116
117 plt.xscale('log')
118 plt.xlabel("Threshold")
119 plt.ylabel("Iterations")
120 plt.legend()
121 plt.savefig("time_comp3")
122 plt.show()
```